

## Лабораторная работа № 2

### Тема: Маршрутизация и HTTP-методы в Express.js

**Цель:** Научиться создавать маршруты Express для разных HTTP-методов, понимать связь между HTTP-методом, URL и обработчиком, а также проверять работу REST API с помощью HTTP-клиента.

#### Практическое задание

Продолжить разработку проекта **Internet Shop API**, начатого в первой лабораторной работе.

Необходимо создать набор маршрутов для ресурсов:

- товары;
- категории;
- заказы.

Основное внимание в этой работе уделяется **маршрутизации и HTTP-методам**. Обработка данных из req.body, параметров пути и query-параметров будет добавлена в следующей лабораторной работе.

#### Последовательность выполнения:

##### 1. Откройте проект express-shop из предыдущей лабораторной работы.

Убедитесь, что зависимости установлены:

`npm install`

Запустите сервер:

`npm start`

Проверьте:

`http://localhost:3000`

##### 2. Оставьте массив товаров из предыдущей лабораторной работы.

Например:

```
const products = [  
  {  
    id: 1,  
    name: "Keyboard",  
    price: 15000,  
    category: "Accessories",  
    inStock: true  
  },  
  {  
    id: 2,  
    name: "Mouse",  
    price: 8000,  
    category: "Accessories",  
    inStock: true  
  }  
]
```

```
];
```

Количество товаров — не менее пяти.

### 3. Реализуйте GET-маршрут получения товаров.

Запрос:

GET /api/products

должен возвращать массив products.

Пример обработчика:

```
app.get("/api/products", (req, res) => {  
  res.json(products);  
});
```

Проверьте маршрут в браузере и HTTP-клиенте.

### 4. Добавьте POST-маршрут.

Создайте:

**POST /api/products**

На данном этапе маршрут пока не должен получать данные нового товара.

Он должен вернуть:

```
{  
  "message": "POST route: product creation"  
}
```

Установите HTTP-статус:

**201 Created**

Таким образом вы проверите, что Express отличает:

GET /api/products

от:

POST /api/products

при одинаковом URL.

Для POST-запросов можно использовать HTTP-client, например Thunder Client (Расширение VSC)

### 5. Добавьте PUT-маршрут.

Создайте:

PUT /api/products

Ответ:

```
{  
  "message": "PUT route: full product replacement"  
}
```

Статус ответа:

200 OK

### 6. Добавьте PATCH-маршрут.

Создайте:

PATCH /api/products

Ответ:

```
{
  "message": "PATCH route: partial product update"
}
```

## 7. Добавьте DELETE-маршрут.

Создайте:

DELETE /api/products

Ответ:

```
{
  "message": "DELETE route: product deletion"
}
```

На этом этапе реально удалять объект из массива не требуется.

## 8. Проверьте пять маршрутов одного ресурса.

Используя Postman, Bruno, Thunder Client или другой HTTP-клиент, последовательно выполните:

GET /api/products

POST /api/products

PUT /api/products

PATCH /api/products

DELETE /api/products

Для всех запросов используйте адрес:

<http://localhost:3000/api/products>

Обратите внимание: **URL остаётся одинаковым, но Express запускает разные обработчики в зависимости от HTTP-метода.**

## 9. Создайте маршруты ресурса categories.

Реализуйте:

GET /api/categories

POST /api/categories

GET должен возвращать массив категорий из предыдущей лабораторной работы.

POST должен возвращать:

```
{
  "message": "POST route: category creation"
}
```

## 10. Создайте маршруты ресурса orders.

Реализуйте:

GET /api/orders

POST /api/orders

Для GET пока можно вернуть пустой массив:

```
[]
```

POST должен вернуть:

```
{
  "message": "POST route: order creation"
}
```

```
}
```

со статусом:  
201 Created

### **11. Вынесите минимум один обработчик в отдельную именованную функцию.**

Например:

```
function getProducts(req, res) {  
  res.json(products);  
}
```

```
app.get("/api/products", getProducts);
```

Остальные обработчики пока можно оставить непосредственно в маршрутах.

Объясните разницу между:

```
app.get("/api/products", (req, res) => {  
  res.json(products);  
});
```

и:

```
app.get("/api/products", getProducts);
```

### **12. Создайте информационный маршрут API.**

Добавьте:

GET /api

Он должен возвращать:

```
{  
  "name": "Internet Shop API",  
  "version": "1.0",  
  "resources": [  
    "/api/products",  
    "/api/categories",  
    "/api/orders"  
  ]  
}
```

### **13. Проверьте несовпадение HTTP-методов.**

Если у вас существует:

POST /api/orders

но отсутствует:

DELETE /api/orders

отправьте DELETE-запрос на:

http://localhost:3000/api/orders

Посмотрите, какой ответ вернёт сервер.

Сделайте вывод: наличие маршрута с одним HTTP-методом не означает наличие маршрута с другим методом для того же URL.

## Итоговый набор обязательных маршрутов

| Метод  | URL             | Назначение                   |
|--------|-----------------|------------------------------|
| GET    | /               | Главная информация           |
| GET    | /api            | Информация об API            |
| GET    | /api/products   | Получение товаров            |
| POST   | /api/products   | Маршрут создания товара      |
| PUT    | /api/products   | Маршрут полной замены товара |
| PATCH  | /api/products   | Маршрут частичного изменения |
| DELETE | /api/products   | Маршрут удаления             |
| GET    | /api/categories | Получение категорий          |
| POST   | /api/categories | Маршрут создания категории   |
| GET    | /api/orders     | Получение заказов            |
| POST   | /api/orders     | Маршрут создания заказа      |

## Требования к результату

Приложение должно:

- запускаться командой `npm start`;
- использовать `Express.js`;
- содержать маршруты с методами GET, POST, PUT, PATCH и DELETE;
- иметь минимум три ресурса: `products`, `categories`, `orders`;
- использовать один и тот же URL `/api/products` с пятью различными HTTP-методами;
- возвращать JSON-ответы;
- использовать корректные HTTP-статусы для успешных запросов;
- содержать минимум один именованный обработчик маршрута;
- корректно работать при проверке через HTTP-клиент.